كلية الهندسة بشبرا
FACULTY OF ENGINEERING AT SHOUBRA

DATA STRUCTURE

MIPS Assembly Expression Evaluation Implementation

"Using Stack Data Structure"

Computer Organization

Team Members

Group Members	B.N
Ahmed Bahy Youssef	2
Ahmed Hossam Mohamed Fekry	3
Ahmed Ragab Mahmoud	4
Ahmed Sobhy Refaay	5
Hassan Hussien Azmy	15
Abdelrahman Mahmoud Mohamed	34

Table of content

Introduction

In computer science, expression parsing, and manipulation play a fundamental role in various applications ranging from compilers to calculators. One crucial transformation is converting an infix expression, where operators are placed between operands, to a postfix expression, also known as Reverse Polish Notation (RPN), where operators follow their operands.

This project focuses on implementing an infix to postfix conversion algorithm using stacks in MIPS assembly language. MIPS assembly, a RISC (Reduced Instruction Set Computing) architecture, provides a platform to delve into low-level programming concepts, including stack manipulation and character processing.

The primary goal of this project is to develop an efficient algorithm to parse infix expressions and generate equivalent postfix expressions while preserving operator precedence rules. This involves managing two stacks: one for operators and another for operands and processing each character of the infix expression accordingly.

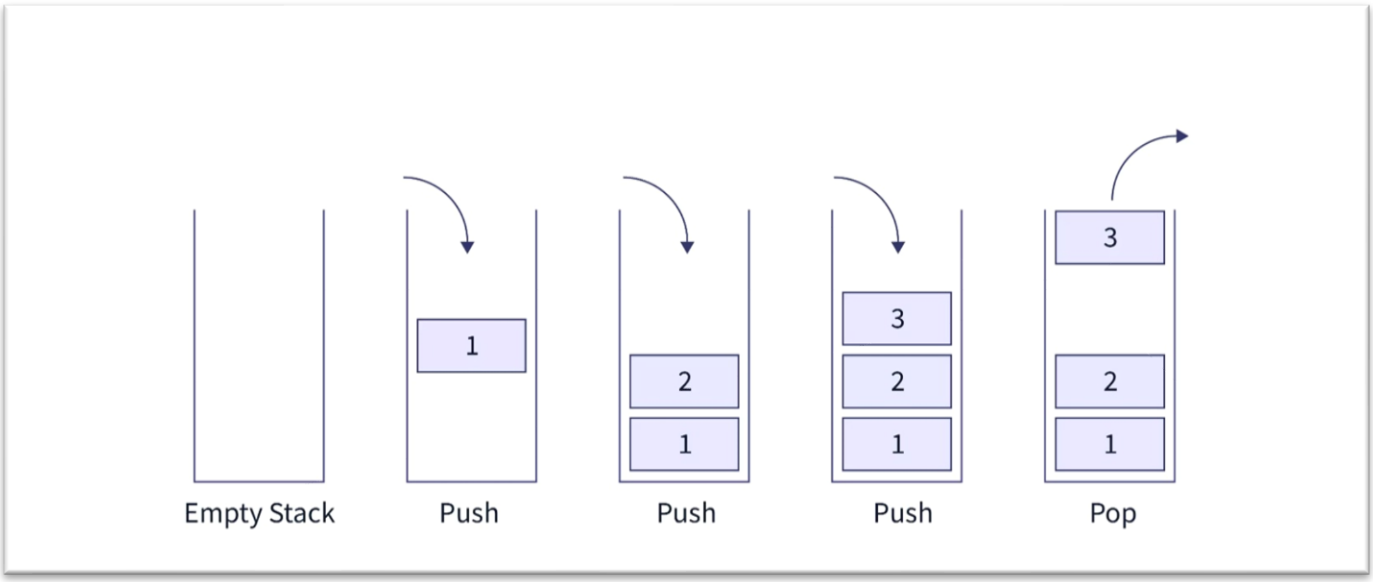
By embarking on this project, we aim to deepen our understanding of stack-based algorithms, character manipulation, and the MIPS assembly language. Through practical implementation and experimentation, we will gain insights into the intricacies of expression parsing and enhance our proficiency in low-level programming paradigms.

Stack Data Structure

A **Stack** is a linear data structure that follows a particular order in which the operations are performed. The order may be **LIFO (Last In First Out)** or **FILO(First In Last Out)**. **LIFO** implies that the element that is inserted last, comes out first and **FILO** implies that the element that is inserted first, comes out last.

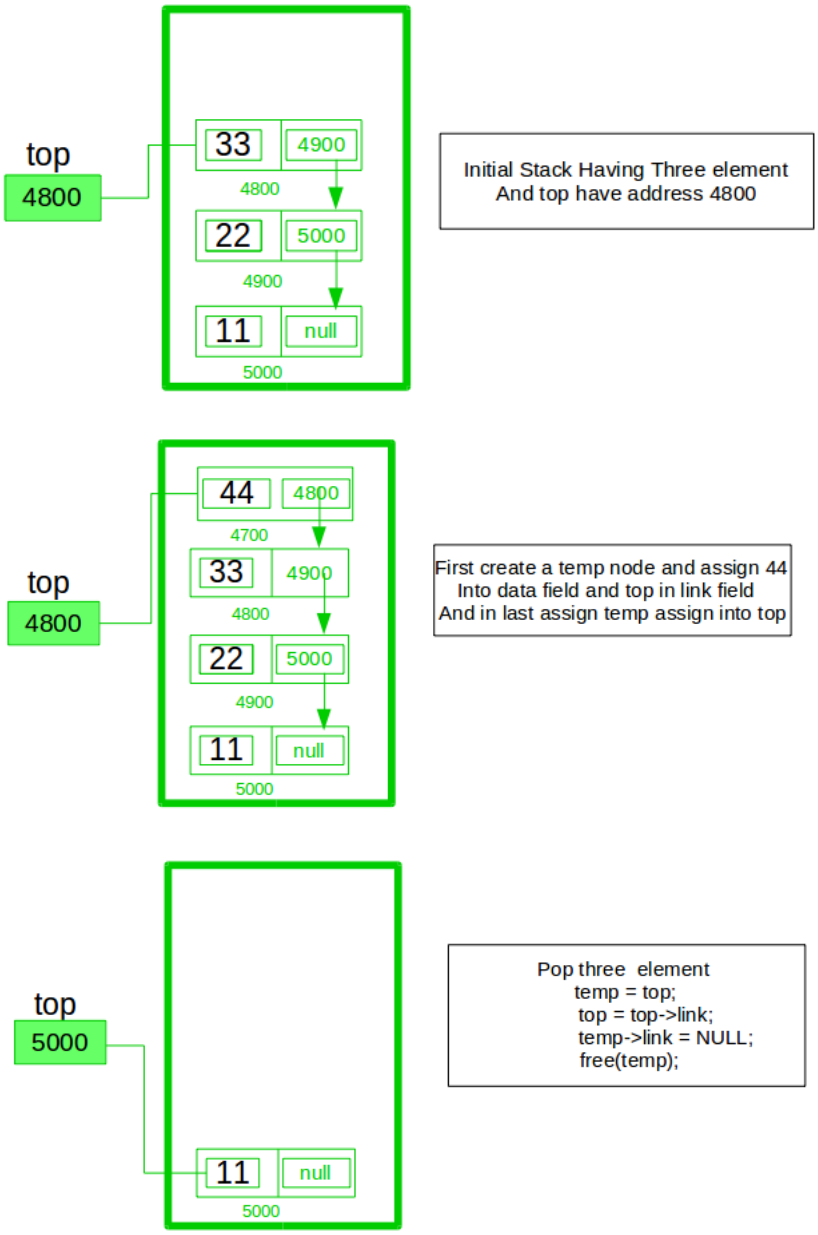
Key Operations on Stack Data Structures

- **Push**: Adds an element to the top of the stack.
- **Pop**: Removes the top element from the stack.
- **Top**: Returns the top element without removing it.
- **IsEmpty**: Checks if the stack is empty.
- **IsFull**: Checks if the stack is full.



Implement a stack using a singly linked list

To implement a stack using the singly linked list concept, all the singly linked operations should be performed based on Stack operations LIFO(last in first out), and with the help of that knowledge, we are going to implement a stack using a singly linked list.



In the stack Implementation, a stack contains a top pointer. which is the “head” of the stack were pushing and popping items happen at the head of the list. The first node has a null in the link field and second node-link has the first node address in the link field and so on and the last node address is in the “top” pointer.

The main advantage of using a linked list over arrays is that it is possible to implement a stack that can shrink or grow as much as needed. Using an array will put a restriction on the maximum capacity of the array which can lead to stack overflow. Here each new node will be dynamically allocated. so, overflow is not possible.

Implementation

Why postfix representation of the expression?

The compiler scans the expression either from left to right or from right to left.

Consider the expression: **a + b * c + d**

- The compiler first scans the expression to evaluate the expression b * c, then again scans the expression to add a to it.
- The result is then added to d after another scan.

The repeated scanning makes it very inefficient. Infix expressions are easily readable and solvable by humans whereas the computer cannot differentiate the operators and parenthesis easily so, it is better to convert the expression to postfix (or prefix) form before evaluation.

The corresponding expression in postfix form is **abc*+d+**. The postfix expressions can be evaluated easily using a stack.

Infix to Postfix Conversion

Expression = A + B * C / D - F + A ^ E

Scanned Symbol	Stack	Output	Reason	
A		A	Step 2	
+	+	A	Step 3.1	
B	+	AB	Step 2	
*	+*	AB	Step 3.1	
C	+*	ABC	Step 2	
/	+/	ABC*	Step 3.2	/ prec. is equal to * so not higher, so going to step 3.2
D	+/	ABC*D	Step 2	
-	-	ABC*D/+	Step 3.2	/ will be popped, added to o/p & then + popped & o/p. - will be pushed
F	-	ABC*D/+F	Step 2	
+	+	ABC*D/+F-	Step 3.2	- will be popped, added to o/p and then + to stack
A	+	ABC*D/+F-A	Step 2	
^	+^	ABC*D/+F-A	Step 2	
E	+^	ABC*D/+F-AE	Step 2	
(empty)		ABC*D/+F-AE^+	Step 8	

How to convert an Infix to a Postfix expression?

*To convert infix expression to postfix expression, use the **stack data structure**. Scan the infix expression from left to right. Whenever we get an operand, add it to the postfix expression, and if we get an operator or parenthesis add it to the stack by maintaining their precedence.*

Below are the steps to implement the above idea:

1. Scan the infix expression **from left to right**.
2. If the scanned character is an operand, put it in the postfix expression.
3. Otherwise, do the following
    - If the precedence and associativity of the scanned operator are greater than the precedence and associativity of the operator in the top stack [or the stack is empty or the stack contains a '**(**'], then push it in the stack. ['^' operator is right associative and other operators like '**+**','**–**','*****' and '**/**' are left-associative].
        - Check especially for a condition when the operator at the top of the stack and the scanned operator both are '^'. In this condition, the precedence of the scanned operator is higher due to its right associativity. So it will be pushed into the operator stack.
        - In all the other cases when the top of the operator stack is the same as the scanned operator, then pop the operator from the stack because of left associativity due to which the scanned operator has less precedence.
    - Else, Pop all the operators from the stack which are greater than or equal to in precedence than that of the scanned operator.
        - After doing that Push the scanned operator to the stack. (If you encounter parenthesis while popping then stop there and push the scanned operator in the stack.)
4. If the scanned character is a '**(**', push it to the stack.
5. If the scanned character is a '**)**', pop the stack and output it until a '**(**' is encountered, and discard both the parenthesis.
6. Repeat steps **2-5** until the infix expression is scanned.
7. Once the scanning is over, Pop the stack and add the operators in the postfix expression until it is not empty.
8. Finally, print the postfix expression.

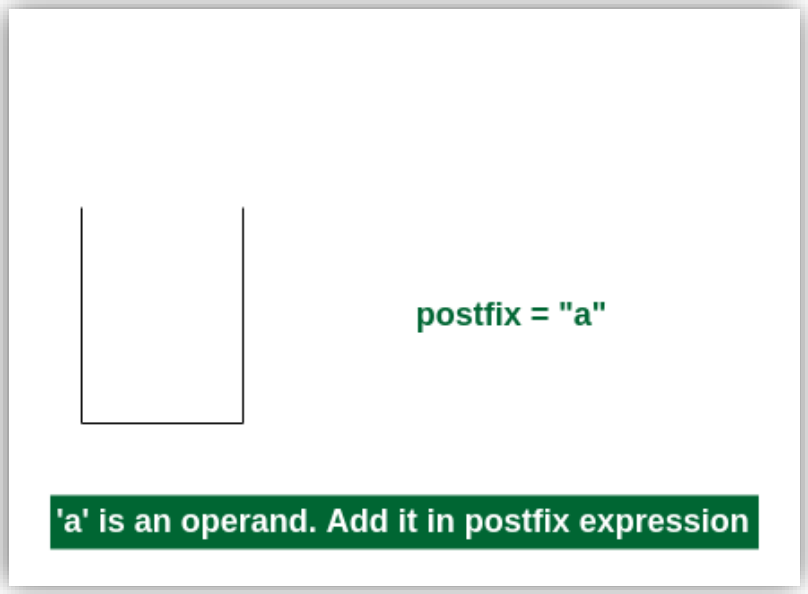
Illustration:

Follow the below illustration for a better understanding

Consider the infix expression ***exp = "a+b*c+d"***

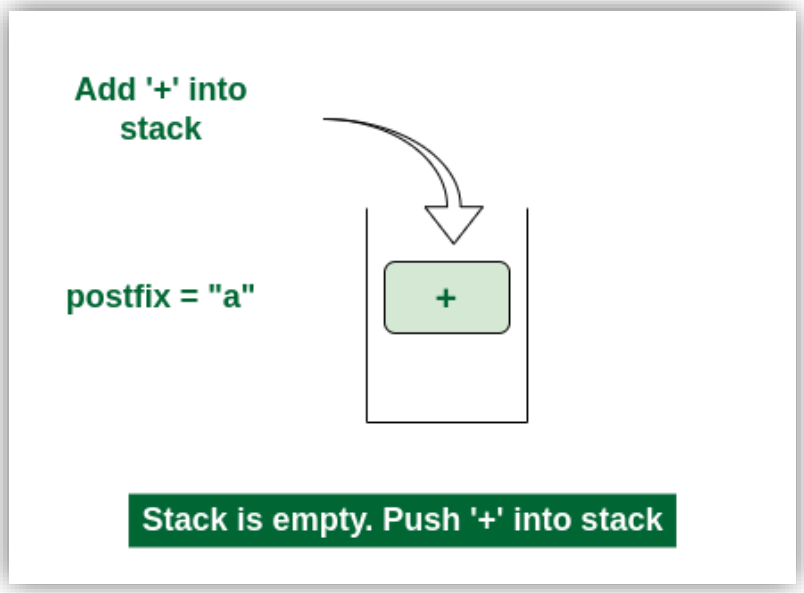
and the infix expression is scanned using the iterator ***i****, which is initialized as* ***i = 0****.*

1st Step: Here i = 0 and exp[i] = 'a' i.e., an operand. So add this in the postfix expression. Therefore, postfix = "a".



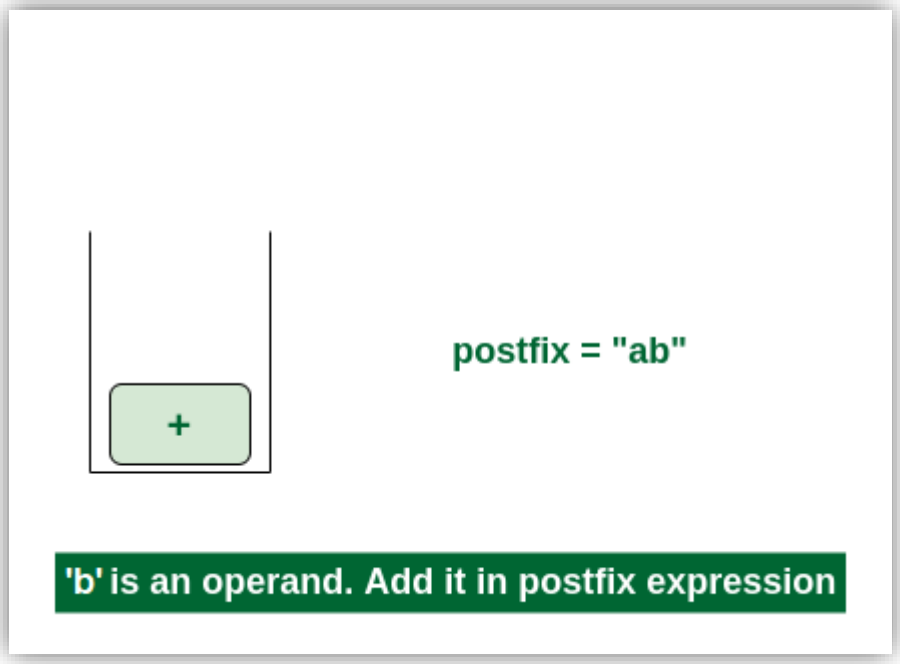
Add 'a' in the postfix

2nd Step: *Here i = 1 and exp[i] = '+' i.e., an operator. Push this into the stack.* ***postfix = "a"*** *and* ***stack = {+}****.*



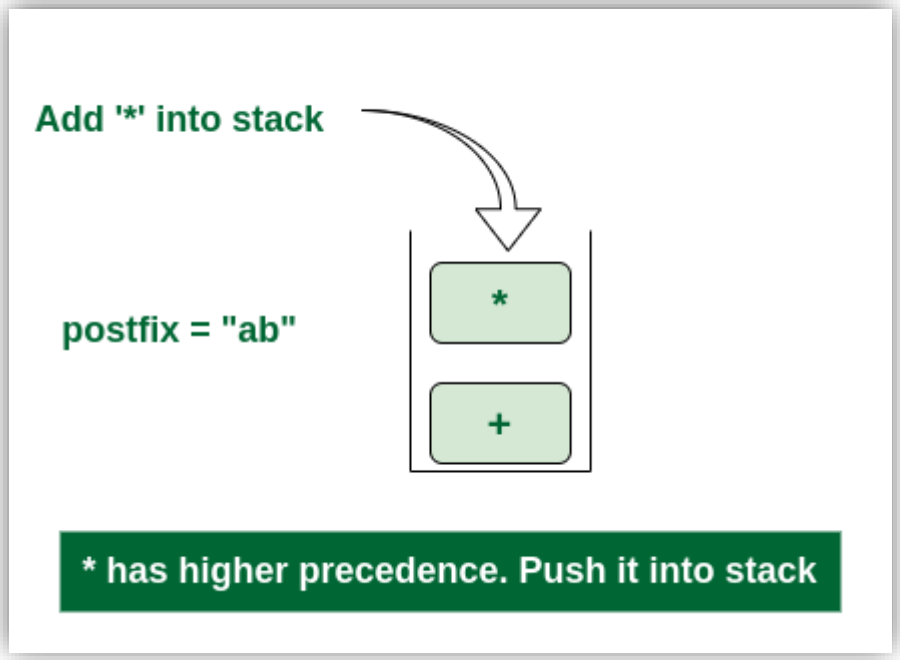
Push '+' in the stack.

3rd Step: Now i = 2 and exp[i] = 'b' i.e., an operand. So add this in the postfix expression. postfix = "ab" and stack = {+}.



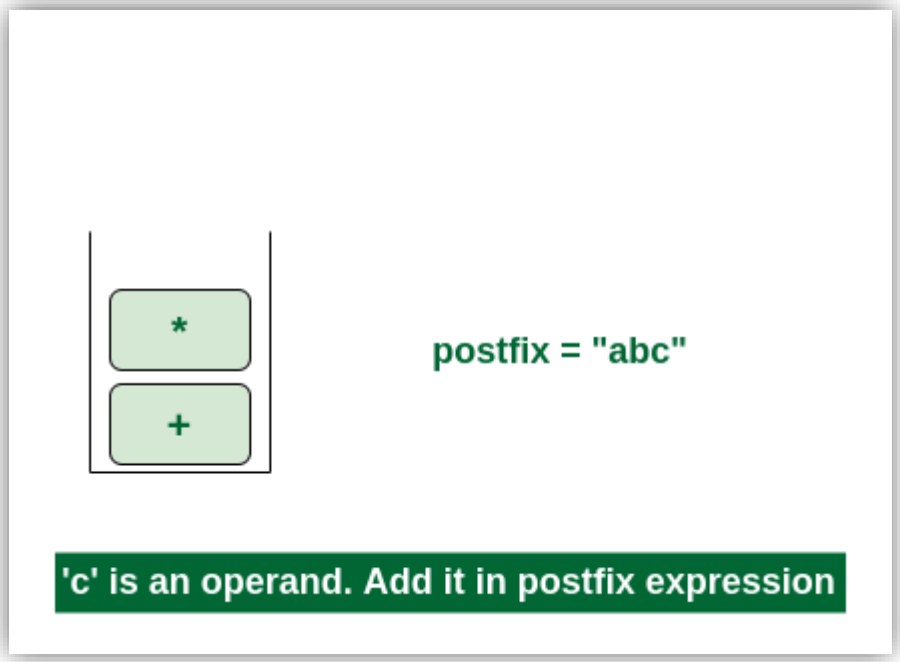
Add 'b' in the postfix

4th Step: *Now i = 3 and exp[i] = '*' i.e., an operator. Push this into the stack.* ***postfix = "ab"*** *and* ***stack = {+, *}.***



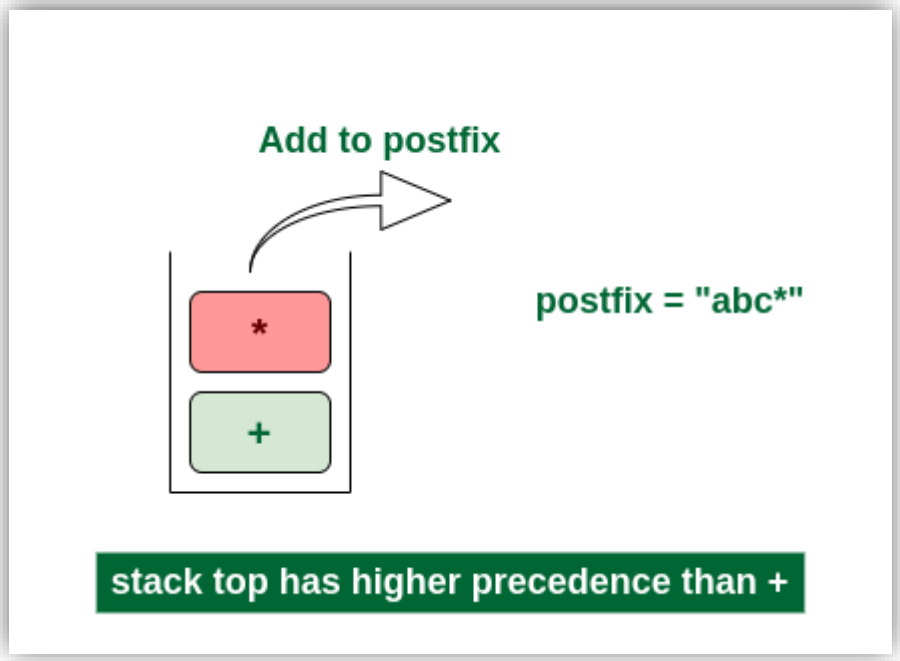
Push '' in the stack.*

5th Step: Now i = 4 and exp[i] = 'c' i.e., an operand. Add this in the postfix expression. postfix = "abc" and stack = {+, *}.



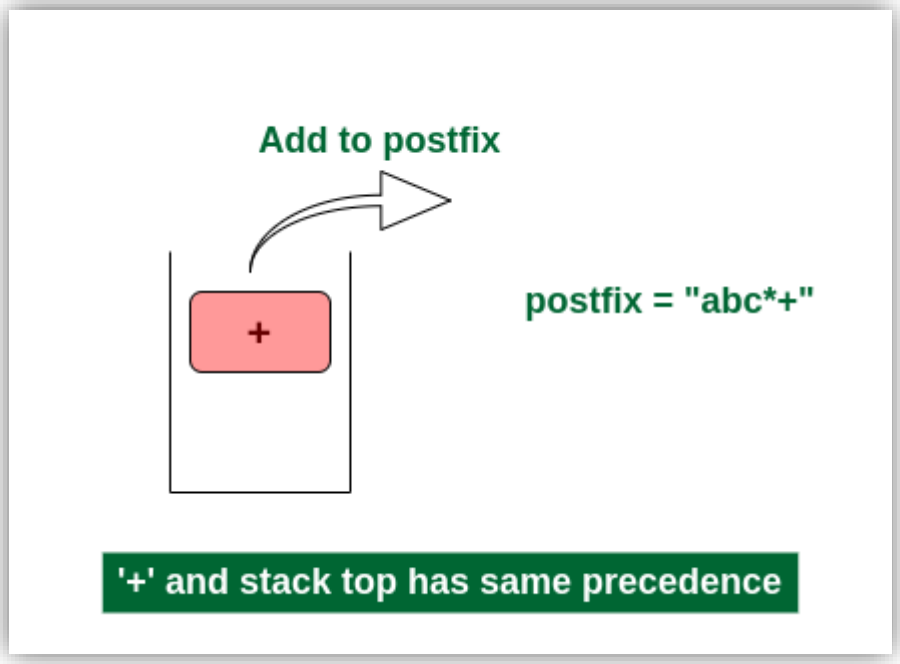
Add ‘c’ in the postfix.

***6th Step:** Now i = 5 and exp[i] = ‘+’ i.e., an operator. The topmost element of the stack has higher precedence. So pop until the stack becomes empty or the top element has less precedence. ‘*’ is popped and added in postfix. So postfix = “abc*” and stack = {+}.*



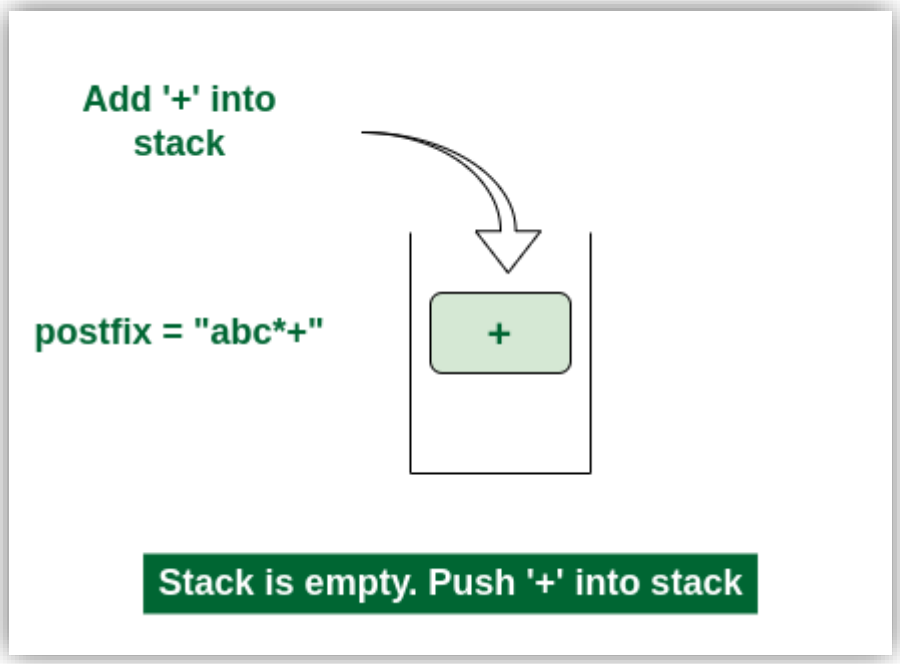
Pop ‘’ and add in postfix.*

Now top element is ‘+‘ that also doesn’t have less precedence. Pop it. postfix = “abc*+”.



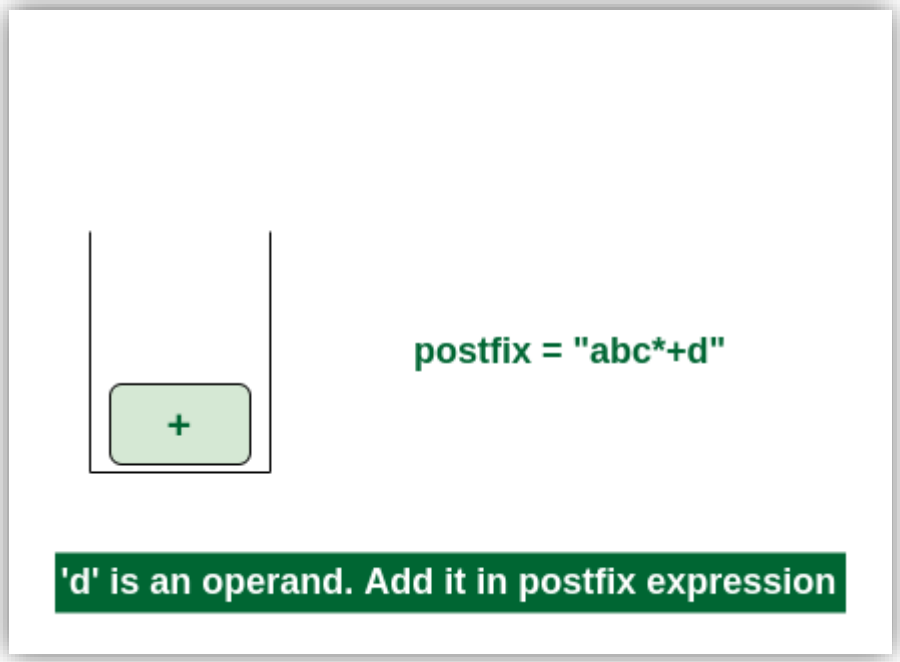
Pop '+' and add it in postfix.

Now stack is empty. So push '+' in the stack. stack = {+}.



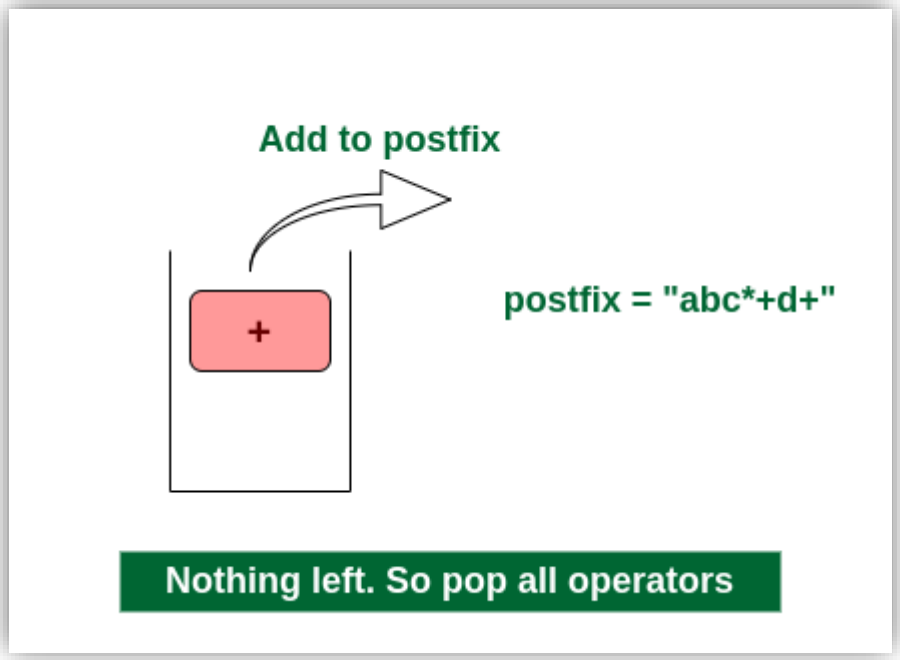
Push '+' in the stack.

7th Step: Now i = 6 and exp[i] = 'd' i.e., an operand. Add this in the postfix expression. postfix = "abc*+d".



Add 'd' in the postfix.

Final Step: Now no element is left. So, empty the stack and add it in the postfix expression. postfix = "abc*+d+".



Pop '+' and add it in postfix.

2 Evaluation

. Using postfix notation, operators are placed after their operands.

. Use a stack to store operands and intermediate results.
. Iterate through each symbol in the expression:
    >> If it's an operand, push it onto the stack.
    >> If it's an operator, pop 2 operands from the stack, apply the operator, and push the result back onto the stack.
After processing all symbols, the final result will be the only element remaining on the stack.

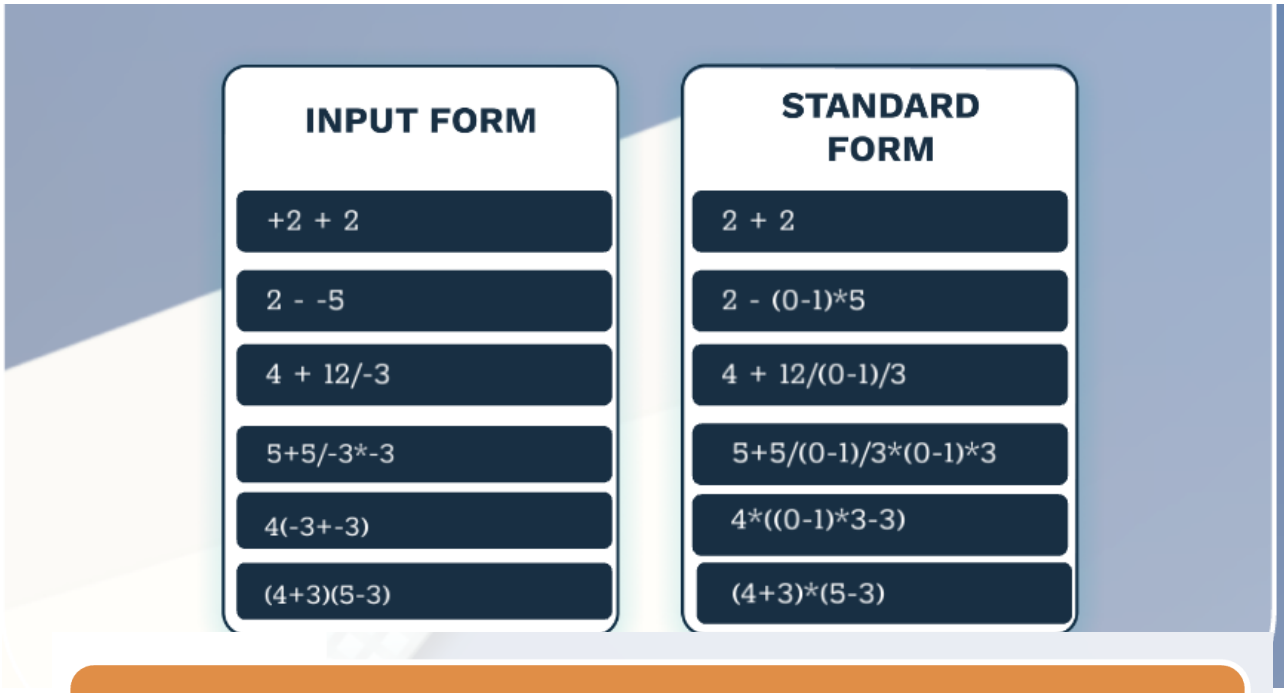
3 Validation & Standard Form

What about 4-3/-3*-(5+3)(3/-2) ?

So,We need convert into **Standard Form**

Handeled Cases

-If "-" follows an operator, adds "**(0-1)***" between them
-If "-" follows "(" or "*", adds "**(0-1)***" between them
-If "-" follows "/" , adds "**(0-1)***" between them
-Adds "*****" before "**(**" if it follows a digit or another "**)**"
-Skips adding "+" if it follows another operator or a "(" directly or came in the front of expression



Valid Expression Rules

- Be Balance.
- Must start with open parenthes or with digit.
- Operators must have operands on both sides.
- No mismatched parentheses or operands

Balance

- Ensures that for every opening parenthesis, there is a corresponding closing parenthesis, and vice versa.

-Examples of unbalnced expressions:

)()) ())()((()(

-Examples of balanced expressions

()() ((()())()) ())(()()

Code Screenshots

1. Stack operations & Creation: -

Push

Mips

```
#a0 = value to be push
push:
    # use $t5 , $t0 , $t1
    addi $sp , $sp , -12
    sw $t0 , 0 ($sp)
    sw $t1 , 4 ($sp)
    sw $t2 , 8 ($sp)
    add $t2,$zero,$a0 # assign the value to be pushed in $t2

    # System call for memory allocation
    li $v0, 9
    # Allocate memory for 2 words (node structure)
    li $a0, 8
    # Allocate memory and store address in $v0
    syscall

    #move $a0 , $t2
    # Store the data value in the node's 'value' field
    sw $t2, 0($v0)

    # Link the new node to the current head of the linked list
    # Load the current head pointer
    lw $t0, head
    # Store the address of the current head in 'next' field of the new node
    sw $t0, 4($v0)
    # Update the head pointer to point to the new node
    sw $v0, head

    # Increment the stack size
    # Load the current stack size
    lw $t1, stack_size
    # Increment the stack size
    addi $t1, $t1, 1
    # Store the updated stack size
    sw $t1, stack_size

    lw $t0 , 0 ($sp)
    lw $t1 , 4 ($sp)
    lw $t2 , 8 ($sp)
    addi $sp , $sp, 12
    jr $ra
```

C++

```
void push(int data)
{

    // Create new node temp and allocate memory in heap
    Node* temp = new Node(data);

    // Initialize data into temp data field
    temp->data = data;

    // Put top pointer reference into temp link
    temp->link = top;

    // Make temp as top of Stack
    top = temp;
    SIZE++ ;
}
```

POP

Mips

```
# Pop operation to remove a node from the stack
pop:
   #use  #$t0 , $t2  and save $ra
   addi $sp , $sp , -12
   sw $ra, 0($sp)
   sw $t0 ,4($sp)
   sw $t2 , 8($sp)

   #lw $t0, stack_size
   lw $t0, head
   # Check if the stack is empty (head is null)
   beq $t0, $zero, pop_empty

   # Update the head pointer to point to the next node
   # Load the address of the next node
   lw $t0, 4($t0)
   # Update the head pointer
   sw $t0, head

   # Decrement the stack size
   # Load the current stack size
   lw $t2, stack_size
   # Decrement the stack size
   addi $t2, $t2, -1
   # Store the updated stack size
   sw $t2, stack_size

   j exitePop
   pop_empty:
      # Stack is empty, handle error or return safely
      la $a0, emptyStack
      jal printString
   exitePop:
      lw $ra, 0($sp)
      lw $t0 ,4($sp)
      lw $t2 , 8($sp)
      addi $sp , $sp , 12
      jr $ra  # return to the caller (evaluation function )
```

C++

```
void pop()
    {
        Node* temp;

        // Check for stack underflow
        if (top == NULL) {
            cout << "\nStack Underflow" << endl;
            exit(1);
        }
        else {

            // Assign top to temp
            temp = top;

            // Assign second node to top
            top = top->link;
            SIZE--;

            // This will automatically destroy
            // the link between first node and second node

            // Release memory of top node
            // i.e delete the node
            free(temp);
        }
    }
```

Print

Mips

```
# function to print stack elements
print:
    addi $sp, $sp, -8
    sw $ra, 0($sp)
    sw $t0 , 4($sp)

    lw $t0, head               # the top pointer
    bne $t0, $zero, loopPrint          # check if it's empty

    la $a0, emptyStack            # message
    jal printString

    j exitLoopPrint
    loopPrint:
       # check if the pointer reached null
       beq $t0, $zero, exitLoopPrint
       # printing the current value
       lw $a0, 0($t0)
       jal printInteger
       jal printNewLine
       # moving to the next node
       lw $t0, 4($t0)
       j loopPrint
    exitLoopPrint:
       lw $ra, 0($sp)
       lw $t0 , 4($sp)
       addi $sp, $sp, 8
       jr $ra
```

C++

```
88
89  // Function to print all the
90  // elements of the stack
91  void print()
92  {
93      Node* temp;
94
95      // Check for stack underflow
96      if (top == NULL) {
97          cout << "\nStack Underflow";
98          exit(1);
99      }
100     else {
101         temp = top;
102         while (temp != NULL) {
103
104             // Print node data
105             cout << temp->data;
106
107             // Assign temp link to temp
108             temp = temp->link;
109             if (temp != NULL)
110                 cout << " -> ";
111         }
112     }
113 }
```

Top

Mips

```
# Top operation to return the value at the top of the stack
top:
    # lw $t0, stack_size
    # use $t0 so save it to the Memory
    addi $sp , $sp , -4
    sw $t0 , 0 ($sp)

    lw $t0, head
    # Check if the stack is empty (head is null)
    beq $t0, $zero, top_empty
    # Load the value stored in the top node
    lw $v0, 0($t0)
    # Return to the caller
    j exit_top

    top_empty:
       # Stack is empty, handle error or return safely
       # Return -1 (or set to an appropriate error value)
       li $v0, -1

    exit_top:
       lw $t0 , 0 ($sp)
       addi $sp , $sp , 4
       jr $ra
```

C

```
51  // Utility function to return top element in a stack
52  int TOP()
53  {
54      // If stack is not empty , return the top element
55      if (!isEmpty())
56          return top->data;
57      else
58          exit(1);
59  }
```

Size

Mips

```
1159  # Get the size of the stack
1160  size:
1161      # Load the stack size
1162      lw $v0, stack_size
1163      jr $ra
```

C++

```
123  int size()
124  {
125      return SIZE ;
126  }
```

FREE

Mips

```
1198  # function to delete all element to the stack
1199  free:
1200     # it use $t0 so save it to the memory
1201     addi $sp , $sp , -4
1202     sw $t0 , 0 ($sp )
1203     #load the head to $t0
1204     lw $t0, head
1205     #loop untill $t0 = 0 (first head )
1206     loopEmpty:
1207        beq $t0, $zero, exitLoopEmpty
1208        lw $t0, 4($t0)
1209        j loopEmpty
1210     exitLoopEmpty:
1211        sw $t0, head
1212        lw $t0 , 0 ($sp)
1213        addi $sp , $sp ,4
1214        jr $ra
```

IsEmpty

Mips

```
1190  isEmpty:
1191     # Load the head pointer
1192     lw $v0, head
1193     # Set $v0 to 1 if head is null
1194     #otherwise 0
1195     seq $v0, $v0, $zero
1196     jr $ra
```

C++

```
42  // Utility function to check if
43  // the stack is empty or not
44  bool isEmpty()
45  {
46      // If top is NULL it means that
47      // there are no elements are in stack
48      return top == NULL;
49  }
```

2. Infix To Postfix Algorithm: -

Infix2Postfix

C++

```cpp

vector<string> infix_to_postfix(string exp)
{
  stack<char> stk;
  vector<string> output;
  int temp1 = 0;
  for (int i = 0; i < exp.length(); i++)
  {
    if (exp[i] == ' ')
      continue;
    if (isdigit(exp[i]))
    {
      while (isdigit(exp[i]))
      {
        int c = exp[i] - '0';
        temp1 = temp1 * 10 + c;
        i++;
      }
      i--;
      string val = to_string(temp1);
      output.push_back(val);
      temp1 = 0;
    }
    else if (exp[i] == '(')
      stk.push('(');
    else if (exp[i] == ')')
    {
      while (stk.top() != '(')
      {
        string xx = string(1, stk.top());
        output.push_back(xx);
        stk.pop();
      }
      stk.pop();
    }
    else
    {
      while (!stk.empty() && Priority(exp[i]) <= Priority(stk.top()))
      {
        if (exp[i] == '^' && stk.top() == '^')
          break;
        string xx = string(1, stk.top());
        output.push_back(xx);
        stk.pop();
      }
      stk.push(exp[i]);
    }
  }
  while (!stk.empty())
  {
    string xx = string(1, stk.top());
    output.push_back(xx);
    stk.pop();
  }
  return output;
}

```

pushToString

```
718  # void pushToString(char c);
719  pushToString:
720      # index where we will push the char
721      lw $t0, tempStringSize
722      # tempString[size] = c
723      sb $a1, tempString($t0)
724      # size ++
725      addi $t0, $t0, 1
726      # updating the size of the tempString
727      sw $t0, tempStringSize
728      jr $ra
```

PushString2Array

Mips

```
# void pushStringToArray(string str);
pushStringToArray:
    addi $sp, $sp, -4
    sw $ra, 0($sp)

    # last index of the tempString
    lw $t0, tempStringSize
    # pushing a null terminator
    sb $zero, tempString($t0)

    # Allocating memory
    # allocating a memory whose address will be stored in $v0
    li $v0, 9
    # size of the memory allocated (10 bytes)
    li $a0, 10
    syscall
    # $t2 = address for the memory allocated
    move $t9, $v0

    # copyString 1st's argument
    la $a1, tempString
    # copyString 2nd's argument
    move $a2, $t9
    # copyString(str1, str2)
    jal copyString
```

```
    # $t3 = array size
    lw $t3, array_size
    # offset = $t3 * 4
    sll $t3, $t3, 2
    # pushing the allocated memory address in the array
    sw $t9, string_array($t3)

    # $t3 = array size
    lw $t3, array_size
    # incrementing the size
    addi $t3, $t3, 1
    # updating the size
    sw $t3, array_size

    # $t0 = 0
    li $t0, 0
    # pointer of the tempString to index 0
    sw $t0, tempStringSize

    lw $ra, 0($sp)
    addi $sp, $sp, 4
    jr $ra
```

toString

```
toString:   # String toString(int num)
    addi $sp, $sp, -4
    sw $ra, 0($sp)

    li $t1, 10          # base
    move $t2, $a0       # num
    li $t3, 0           # i
    convert_loop:
        # num / 10
        div $t2, $t1
        # $t9 = num % 10
        mfhi $t9
        # $t9 = char(num)
        addi $t9, $t9, 48
        # storing the char in tempString
        sb $t9, tempString($t3)
        # get the quotient
        mflo $t2
        # increment i
        addi $t3, $t3, 1
        # repeat until quotient is zero
        bnez $t2, convert_loop
        j reverse

    reverse:
        li $t1, 0
        # store the size of the tempString
        sw $t3, tempStringSize
        # the last index of the string
        addi $t3, $t3, -1
```

```
reverse_loop:
    # Exit loop if start index >= end index
    bge $t1, $t3, reverse_done
    # Load character at start index
    lb $t2, tempString($t1)
    # Load character at end index
    lb $t8, tempString($t3)
    # Store character from end index at start index
    sb $t8, tempString($t1)
    # Store character from start index at end index
    sb $t2, tempString($t3)
    # Move start index forward
    # Move end index backward
    addi $t1, $t1, 1
    addi $t3, $t3, -1
    # loop
    j reverse_loop

reverse_done:
    lw $ra, 0($sp)
    addi $sp, $sp, 4
    jr $ra
```

Mips

Periority

```
periority:
    addi $sp, $sp, -4
    sw $t5, 0($sp)

    # ASCII for '+'
    li $t1, '+'
    # ASCII for '-'
    li $t8, '-'
    # ASCII for '*'
    li $t9, '*'
    # ASCII for '/'
    li $t0, '/'
    # ASCII for '^'
    li $t5, '^'

    # Check the input character and return priority
    beq $a0, $t1, plus_minus_label
    beq $a0, $t8, plus_minus_label
    beq $a0, $t9, multiply_division_label
    beq $a0, $t0, multiply_division_label
    beq $a0, $t5, exponential_label

    li $v0, 0
    j returnPriority
```

```
plus_minus_label:
    # Return 1 for '+' or '-'
    li $v0, 1
    j returnPriority

multiply_division_label:
    # Return 2 for '*' or '/'
    li $v0, 2
    j returnPriority

exponential_label:
    # Return 3 for '^'
    li $v0, 3
    j returnPriority

returnPriority:
    lw $t5, 0($sp)
    addi $sp, $sp, 4
    jr $ra
```

C++

```
int Priority(char c)
{
  if (c == '-' || c == '+')
    return 1;
  else if (c == '*' || c == '/')
    return 2;
  else if (c == '^')
    return 3;
  else
    return 0;
}
```

IsDigit

Mips

```
isDigit: # bool isDigit(char c);
        # Set the default value to 0
        li $v0, 0
        # if(exp[i] < '0') return 0
        blt $a1, '0', end_isDigit
        # if(exp[i] > '9') return 0
        bgt $a1, '9', end_isDigit
        # if(exp[i] >= '0' && exp[i] <= '9') return 1
        li $v0, 1

    end_isDigit:
        jr $ra
```

C++

```
isdigit(char ch)
{
  return ch >='0'  && ch <='9' ;
}
```

```
Infix_to_postfix:
    addi $sp, $sp, -4
    sw $ra, 0($sp)

    # temp --> to split the expression
    li $t2, 0
    # counter
    li $t4, 0
    stringLoop:
        # $t5 = expression[i]
        lb $t5, expression($t4)
        # if(expression[i] == '\n') break
        beq $t5, '\n', exitStringLoop
        # if(null-terminated) break
        beq $t5, $zero, exitStringLoop
        # if(expression[i] == ' ') continue
        beq $t5, ' ', increment

        # isDigit's argument
        move $a1, $t5
        # $v0 = isDigit(expression[i])
        jal isDigit
        # if(isOperator(expression[i])) jump
        beq $v0, $zero, operator
```

```
innerLoop:
    # if(expression[i]) == ' ') continue
    beq $t5, ' ', increment
    # $t3 = int(expression[i])
    addi $t3, $t5, -48
    # temp = temp * 10
    mul $t2, $t2, 10
    # temp = temp * 10 + int(exp[i])
    add $t2, $t2, $t3
    # i ++
    addi $t4, $t4, 1

    # loading the next char
    lb $t5, expression($t4)
    # if(expression[i] == '\n') break
    beq $t5, '\n', exitStringLoop
    # if(null terminated) break
    beq $t5, $zero, exitStringLoop

    # isDigit's argument
    move $a1, $t5
    # $v0 = isDigit(expression[i])
    jal isDigit
    # if(!isDigit(exp[i]))  break
    bne $v0, $zero, innerLoop
```

```
exitInnerLoop:
    # i --
    addi $t4, $t4, -1
    # toString's argument
    move $a0, $t2
    # toString(temp)
    jal toString
    # pushStringToArray()
    jal pushStringToArray
    # temp = 0
    li $t2, 0
    # i++ and loop
    j increment
operator:
    beq $t5, '(', leftParenthesis
    beq $t5, ')', rightParenthesis
    bge $t5, 42, else

    # if(expression[i] == '(') push
    leftParenthesis:
        # argument
        move $a0, $t5
        # stackPush(exp[i])
        jal push
        # i++ and loop
        j increment
```

```
rightParenthesis:
    parenthesisLoop:
        # $v0 = stackTop()
        jal top
        # $t6 = $v0
        move $t6, $v0
        # loop until '(' is found
        beq $t6, '(', exitParenthesisLoop

        # pushToString's argument
        move $a1, $t6
        # pushToString(stackTop())
        jal pushToString
        # push the operator to the array
        jal pushStringToArray
        # stackPop()
        jal pop
        # loop
        j parenthesisLoop
    exitParenthesisLoop:
        # stackPop()
        jal pop
        # i++ and loop
        j increment
```

```
else:
    elseLoop:
        # $v0 = stackTop()
        jal top
        # $t6 = $v0
        move $t6, $v0
        # exit if stack is empty
        beq $t6, -1, exitElseLoop

        # if(stackTop() == '^') t0 = 1
        seq $t0, $t6, '^'
        # if(exp[i] == '^') t1 = 1
        seq $t1, $t5, '^'
        # $t1 = $t0 & $t1
        and $t1, $t1, $t0
        # if("^^") push(exp[i])
        beq $t1, 1, exitElseLoop

        # periority's argument
        move $a0, $t5
        # $v0 = periority(exp[i])
        jal periority
        # $t7 = returned value
        move $t7, $v0

        # periority's argument
        move $a0, $t6
        # $v0 = periority(stackTop())
        jal periority
        # pushing the largest priority
        bgt $t7, $v0, exitElseLoop
```

```
                # pushToString's argument
                move $a1, $t6
                # pushToString(stackTop())
                jal pushToString
                # push the operator to the array
                jal pushStringToArray
                # stackPop()
                jal pop
                # loop
                j elseLoop
            exitElseLoop:
                # stackPush's argument
                move $a0, $t5
                # stackPush(exp[i])
                jal push
    increment:
        # i++
        addi $t4, $t4, 1
        # loop
        j stringLoop
exitStringLoop:
    # i-- --> expression.size
    addi $t4, $t4, -1
    # $t1 = expression[size - 1]
    lb $t1, expression($t4)
    # if($t1 == ')') empty the stack
    beq $t1, ')', stackNotEmptyLoop
    # toString's argument
    move $a0, $t2
    # toString(exp[size - 1])
    jal toString
    # pushStringToArray()
    jal pushStringToArray
```

```
625  stackNotEmptyLoop:
626      # $v0 = stackTop()
627      jal top
628      # $t6 = $v0
629      move $t6, $v0
630      # if(stackEmpty()) exit
631      beq $t6, -1, exitStackNotEmptyLoop
632
633      # pushToString's argument
634      move $a1, $t6
635      # pushToString(stackTop())
636      jal pushToString
637      # pushStringToArray()
638      jal pushStringToArray
639      # stackPop()
640      jal pop
641      # loop
642      j stackNotEmptyLoop
643
644  exitStackNotEmptyLoop:
645
646      lw $ra, 0($sp)
647      addi $sp, $sp, 4
648      jr $ra
```

3. Expression Evaluation: -

C++

```cpp

int Evaluation(vector<string>v)
{

    stack <int> st ;
    for (int i =0 ;i<v.size() ;i++)
    {
        if (is_operator(v[i][0]))
        {
                int b = st.top() ;
                st.pop();

                int a = st.top();
                st.pop();
                double val = make_operation(a, b, v[i][0]);
                st.push(val);
        }
        else
        {
            int val = string_to_int(v[i]);
            st.push(val) ;

        }
    }
        return st.top();
}
```

is_operator

Mips

```
1042  isOperator:
1043     addi $sp , $sp , -24
1044     sw $t0, 0($sp)
1045     sw $t1, 4($sp)
1046     sw $t2, 8($sp)
1047     sw $t3, 12($sp)
1048     sw $t4, 16($sp)
1049     sw $t5, 20($sp)
1050
1051     move $v0, $zero
1052     move $t0, $a0
1053     li $t1, '+'
1054     li $t2, '-'
1055     li $t3, '*'
1056     li $t4, '/'
1057     li $t5, '^'
1058
1059     beq $t0, $t1, set
1060     beq $t0, $t2, set
1061     beq $t0, $t3, set
1062     beq $t0, $t4, set
1063     beq $t0, $t5, set
1064     j exit
1065     #exit label change boolean value
1066     set:
1067        addi $v0, $zero, 1
1068     exit:
1069        lw $t0 , 0($sp)
1070        lw $t1 , 4 ($sp)
1071        lw $t2 , 8 ($sp)
1072        lw $t3 , 12 ($sp)
1073        lw $t4 , 16 ($sp)
1074        lw $t5 , 20 ($sp)
1075        addi $sp ,$sp , 24
1076        jr $ra
```

C++

```
81  bool is_operator(char c)
82  {
83    return (c == '+') || (c == '-') || (c == '*') || (c == '/') || (c == '^');
84  }
```

MakeOperation

C++

```cpp
int make_operation(int a, int  b,char op)
{
    int val =0;
    switch (op)
    {
        case '+':
            val = a+b;
            break;
        case '-':
            val = a-b;
            break ;
        case '/':
            val = a/b;
            break;
        case '*':
            val = a*b;
            break;

        case '^':
            val = power(a,b)  ;
        default:
            break;

    }
    return val ;
}
```

Power

Mips

```
power:
   #arguments: base ---> a, exponenet ----> a1      2,3
   # use $t0 , $t1 , $t2
   addi $sp , $sp, -12
   sw $t0, 0($sp)
   sw $t1, 4($sp)
   sw $t2, 8($sp)
   # result t1 = 1
   addi $t1, $0, 1
   # index = 0
   add $t0, $0, $0
   loop1:
      slt $t2, $t0, $a1
      # if index > exponenet ----> exit loop
      beq  $t2, $0, exit_loop1
      #  operation
      mul $t1, $t1, $a0
      # incrementing index
      addi $t0, $t0, 1
      j loop1
   exit_loop1:
      # result in V0
      add $v0, $t1, $0
      lw $t0 , 0($sp)
      lw $t1 , 4 ($sp)
      lw $t2 , 8 ($sp)
      addi  $sp , $sp , 12
      jr $ra
```

C++

```
int power(int a ,int b )
{
    // return a ^b
    int value =1 ;
    for (int i =0; i<b ;i++)
    {
        value *=a ;
    }
    return value;
}
```

MakeOperation

Mips

```
makeoperation:
   #arguments   ---->  operand1 a0,  operand a1 and operation a2

   #use $t0  so save it on the memory
   addi $sp , $sp , -4
   sw $t0 , 0($sp)
   #checking if the operation is +
   li $t0, '+'
   beq $a2, $t0, plus_exit

   #checking if the operation is -
   li $t0, '-'
   beq $a2, $t0, minus_exit

   #checking if the operation is *
   li $t0, '*'
   beq $a2, $t0, mult_exit

   #checking if the operation is /
   li $t0, '/'
   beq $a2, $t0, divison_exit

   #checking if the operation is ^
   li $t0, '^'
   beq $a2, $t0, exponent_exit

   # if the input is invalid
   # flag ig operation is invalid
   add $t1, $a0, $0
   j exiteMakeoperation
```

```
   plus_exit:
      add $v0, $a0, $a1
      j exiteMakeoperation

   minus_exit:
      sub $v0, $a0, $a1
      j exiteMakeoperation

   mult_exit:
      mul $v0, $a0, $a1
      j exiteMakeoperation

   divison_exit:
      div $v0, $a0, $a1
      j exiteMakeoperation

   exponent_exit:
         addi $sp, $sp, -4
         sw   $ra, 0($sp)
         # calling power function
         jal power
         lw   $ra, 0($sp)
         addi $sp, $sp, 4

      j exiteMakeoperation
   exiteMakeoperation:
      lw $t0 , 0($sp)
      addi $sp , $sp, 4
      jr $ra
```

string_to_int

Mips

```
strToInt:
    # use $t0 , $t1  , $t2, $t3  so save them in momory
    addi $sp, $sp, -16
    sw $t0, 0($sp)
    sw $t1, 4($sp)
    sw $t2, 8($sp)
    sw $t3, 12($sp)

    # Loop counter i
    li $t0, 0
    # Initialize ans to 0
    li $t1, 0
    strToInt_loop:
       # Load the byte (character) from the string into $t2
       lb $t2, 0($a0)
       # If the byte is 0 (null terminator), exit loop
       beqz $t2, strToInt_done
       # ASCII value of '0'
       li $t3, 48
       sub $t2, $t2, $t3
       # Multiply ans by 10
       mul $t1, $t1, 10
       # Add the converted character to ans
       add $t1, $t1, $t2
       # Increment loop counter and pointer to next character
       addi $t0, $t0, 1
       addi $a0, $a0, 1
       # Repeat the loop
       j strToInt_loop

    strToInt_done:
       # Move the result to $v0
       move $v0, $t1
       lw $t0 , 0($sp)
       lw $t1 , 4 ($sp)
       lw $t2 , 8 ($sp)
       lw $t3 , 12 ($sp)
       addi $sp  , $sp, 16
       jr $ra
```

C++

```
84
85  long long string_to_int(string s)
86  {
87      long long val =0 ;
88      for (char i:s)
89      {
90          val= val *10 + (i-'0') ;
91      }
92      return val;
93  }
94
```

Evaluation

Mips

```
Evaluation:
    #a0 address of the refrance array of  strings
    #loop for each string refrance in the array
    #counter i
    li $t7 , 0
    lw $t6 , array_size
    # address of refrance array
    move $s2 , $a0

    # saving the return address in the stack
    addi $sp, $sp, -4
    sw   $ra, 0($sp)
    #move $t8 , $ra
    #ra is the position in the main function

    loop:
       beq $t7 , $t6 , exite_loop

       # load the address of string
       # refrance of string
       lw $a0 , 0 ($s2)
       lb $a0 , 0 ($a0)
       beq $a0 , $zero , increment2
       jal isOperator
       beq $v0 , $0 , Number
```

```
    # get the top in v0
    jal top
    # b
    move $s3 , $v0
    jal pop
    # get the top in v0
    jal top
    # a
    move $s4, $v0
    jal pop
    move $a0 , $s4
    move $a1 , $s3
    lw $t9, 0($s2)
    lb $a2 , 0($t0)
    jal makeoperation
    move $a0 , $v0
    jal push
    increment2:
       addi $s2 , $s2 , 4
       addi $t7  , $t7, 1
       j  loop
    Number:
       # refrance of string to a0
       lw $a0 , 0 ($s2)
       jal strToInt
       move $a0 , $v0
       jal push
       addi $s2 , $s2 , 4
       addi $t7  , $t7, 1
       j  loop

exite_loop:
    # result in $v0
    jal top
    lw   $ra, 0($sp)
    addi $sp, $sp, 4
    jr $ra
```

C++

```
int Evaluation(vector<string>v)
{

    stack <int> st ;
    for (int i =0 ;i<v.size() ;i++)
    {
        if (is_operator(v[i][0]))
        {
                int b = st.top() ;
                st.pop();

                int a = st.top();
                st.pop();
                double val = make_operation(a, b, v[i][0]);
                st.push(val);
        }
        else
        {
            int val = string_to_int(v[i]);
            st.push(val) ;

        }
    }
        return st.top();
}
```

4. Validation & Standard Form: -

.data Section

```
1  .data
2  # Global variables
3      minusMul:                    .asciiz  "(0-1)*"
4      minusDiv:                    .asciiz  "(0-1)/"
```

toStandardInfix

C

```
178
179 string to_standard_infix(string s)
180 {
181   string res = "";
182   if (s[0] == '-')
183     res += "(0-1)*";
184   else if (s[0] != '+')
185     res += s[0];
186
187   for (int i = 1; i < s.size(); i++)
188   {
189     // 5 *+2 = > 5 *2 remove unwanted +ve sign
190     if (s[i] == '+' && is_operator(s[i - 1]))
191       continue;
192     //          7(6) => 7*(6)              ||       (5)(8) = > (5)*(8)
193     if ((s[i] == '(' && isdigit(s[i - 1])) || (s[i] == '(' && s[i - 1] == ')'))
194       res += '*';
195     if (s[i] == '-' && is_operator(s[i - 1]))
196     {
197       if (s[i - 1] == '/')
198         res += "(0-1)/"; //  5*-2 = 5*(0-1)*2
199       else
200         res += "(0-1)*"; //  5/-2 = 5/(0-1)/2
201     }
202     else if (s[i] == '-' && s[i - 1] == '(')
203       res += "(0-1)*"; // (-2) = ((0-1)*2)
204     else
205       res += s[i];
206   }
207   return res;
208 }
```

```
268
269 #-----------------------------------------------------------------------------------------------------------
270 # a1 --> the string to be copied ("(0-1)")
271 # copy loop function
272 Copy_additional_expression:
273
274     addi $sp , $sp , -16
275     sw $t0, 0($sp)
276     sw $t1, 4($sp)
277     sw $t2, 8($sp)
278     sw $t3, 12($sp)
279
280         la  $t0  , res   # copy the refrance of string to be copied in it
281
282     move $t1 , $a1    # copy te refrance of string  to be copied ("(0-1)"
283
284     # s1 contain result size
285     add $t0 , $t0 , $s1   # start index to be stor in it
286
287     Copy_additional_expression_loop:
288
289             lb  $t2, 0($t1)             # Load the first byte of $a1 (str1) into $t2
290             beq  $t2, $zero, done_copy  # if null so copy done and go out of the loop
291             sb   $t2, 0($t0)             # Save the value in $t2 at the same byte in $s1 (str2)
292             addi $t0, $t0, 1             # Increment both memory locations by 1 byte
293             addi $t1, $t1, 1
294             addi $s1 , $s1,1   # increase the size of the result string
295             j  Copy_additional_expression_loop
296
297         done_copy:
298             lw $t0 , 0($sp)
299             lw $t1 , 4 ($sp)
300             lw $t2 , 8 ($sp)
301             lw $t3 , 12 ($sp)
302             addi $sp , $sp , 16
303                 jr $ra
304 #-----------------------------------------------------------------------------------------------------------
305 # isvalid function
306 #a0 , refrance of string
```

Mips

```
150 toStandardInfix:
151     addi   $sp ,$sp ,-4
152     sw $ra , 0($sp)
153
154     # counter i
155     li $t3 , 0
156     # &filtered expression
157     la $s0 , filteredExp
158     # standardExpression size
159     li $s1 ,0
160     # &standardExpression
161     la $s2 , standardExp
162
163     to_standard_infix_loop:
164        # t1 --> s[i]        t0 --> s[i-1]
165        # load from filteredExpression
166        lb $t1 , 0($s0)
167        # if(null-terminator) break
168        beq $t1 , $zero , end_to_standard_loop
169        # if(i != 0) jump
170        bne $zero ,$t3  , I_largThanZero
171        # if(s[0] == '-') push (0-1)*
172        beq $t1, '-', Edit_for_multiplication
173        # if(exp[0] == '+') continue
174        beq $t1 , '+', next_iteration
175        # if(exp[0] != '-' && exp[0] != '+') push
176        j push_to_res
```

```
178 I_largThanZero:
179     # filteredExpression[i-1]
180     lb $t0 , -1($s0)
181     # if(exp[i] != '+') jump
182     bne $t1 , '+' , continue
183
184     # if(exp[i] == '+')
185     # $v0 = isOperator(exp[i-1])
186     move $a0 , $t0
187     jal isOperator
188     # if(exp[i]=='+' && isOperator(exp[i-1])) continue
189     beq $v0,$0, handlerightbracket
190     j next_iteration
191
192     handlerightbracket:
193        # if(exp[i]=='+' && exp[i-1] == '(')   continue
194        beq $t0 , '(' , next_iteration
195        # else
196        j push_to_res
197
198        # if(exp[i] != '+')
199     continue:
200        # if(exp[i] != '(')  jump
201        bne $t1, '(',continue2
202        # if )( --> push '*'
203        beq $t0,')',add_asterisk
```

```
205        # isDigit(s[i-1])
206        move $a1 , $t0
207        jal isDigit
208        # if(!isDigit(s[i-1])) jump
209        beq $v0,$0, continue2
210        # if A( --> push '*'
211        j add_asterisk
212
213        # if(exp[i] != '(' || !isDigit(exp[i-1]))
214     continue2:
215        # if(exp[i] != '-') push s[i]
216        bne $t1, '-', push_to_res
217
218        # if(exp[i] == '-')
219        # $v0 = isOperator(s[i-1])
220        move $a0 , $t0
221        jal isOperator
222        # if(!isOperator(s[i-1])) jump
223        beq $v0, $0, not_operator
224
225        # if(isOperator(s[i-1]))
226        # if /-  --> push (0-1)/
227        beq $t0, '/', Edit_for_division
228        # else  --> push (0-1)*
229        j Edit_for_multiplication
230     not_operator:
231        # if (- --> push (0-1)*
232        beq $t0,'(',Edit_for_multiplication
233        # else
234        j push_to_res
```

```
Edit_for_multiplication:
    # push (0-1)*
    la $a1, minusMul
    jal Copy_additional_expression
    j next_iteration
Edit_for_division:
    # push (0-1)/
    la $a1, minusDiv
    jal Copy_additional_expression
    j next_iteration

add_asterisk:
    # Load the ASCII code for '*'
    li $t4, '*'
    # next standardExp's idx
    add $t8 , $s2 , $s1
    # Storing an *
    sb $t4, 0($t8)
    # increase the size of the standardExp
    addi $s1, $s1 , 1
    # pushing the '('
    j push_to_res

push_to_res:
    # next standardExp idx
    add $t8 , $s2 , $s1
    # store filteredExp[i] in standardExp
    sb $t1, 0($t8)
    # increase the size of the standardExp
    addi $s1, $s1 , 1
```

```
        next_iteration:
            # i ++
            addi $t3, $t3 , 1
            addi $s0, $s0 , 1
            j to_standard_infix_loop
end_to_standard_loop:

    add $t8 , $s2 , $s1
    sb $zero, 0($t8)
    lw $ra , 0($sp)
    addi $sp , $sp , 4
    jr $ra
```

IsValidExpression

c

```
bool isValidExpression(string s)
{
    if(!Balance(s))return false;
    int prev=-1;        // 1 for digits , 2 for operators , 3 for '('  , 4 for ')'
    for(char c:s){
        if(c=='(')
        {
            if(prev==4 || prev==1)return false;
            prev=3;
        }
        else if(c==')')
        {
            if(prev==-1 || prev==3)return false;
            prev=4;
        }else if(isdigit(c))
        {
            if(prev==4)return false;
            prev=1;
        }else if(c=='-'  || c=='+'  || c=='*' || c=='/')
        {
            if(prev==-1 || prev==2 ||  prev==3)return false;
            prev=2;
        }
    }
    if(prev==3 || prev==2)
        return false;
    return true;
}
```

Mips

```
isValidExpression:
    addi $sp, $sp, -16
    sw $ra, 0($sp)
    sw $t0, 4($sp)
    sw $t1, 8($sp)
    sw $t2, 12($sp)

    # &standardExp
    move $t2 , $a0
    # flag for the previous char
    # 3-->digit, 2-->operator
    # 3-->'(',   4-->')'
    li $t0, -1

    IsvalidLoop:
      # load from standardExp
      lb $t1, 0($t2)
      # if('\n') break
      beq $t1, '\n', end_loop
      # if(null-terminator) break
      beqz $t1, end_loop
      # moving to next character
      addi $t2, $t2, 1

      # Check character
      beq $t1, '(', check_open_bracket
      beq $t1, ')', check_close_bracket

      # call isDigit function    $v0 return flage
      move $a1 , $t1
      jal isDigit              # check if character is a digit
      bne $v0, $zero, checkDigit
```

```
    # call is operator function
    move $a0 , $t1
    jal isOperator
    # if not a digit, check if it's an operator
    bne $v0, $zero, check_operator

check_open_bracket:
    # if previous was ')' or initial, return false
    beq $t0, 4, unbalanced
    # if previous was a digit, return false
    beq $t0, 1, unbalanced
    # set prev = 3 (for '(')
    li $t0, 3
    j IsvalidLoop

check_close_bracket:
    # if previous was initial, return false
    beq $t0, -1, unbalanced
    # if previous was '(', return false
    beq $t0, 3, unbalanced
    # if previous was operator , return false
    beq $t0, 2, unbalanced
    # set prev = 4 (for ')')
    li $t0, 4
    j IsvalidLoop
checkDigit:
    beq $t0, 4, unbalanced
    li $t0, 1
    j IsvalidLoop
```

```
check_operator:
  # if previous was initial, return false
  beq $t0, -1, unbalanced
  # if previous was an operator, return false
  beq $t0, 2, unbalanced
  # if previous was '(', return false
  beq $t0, 3, unbalanced
  # set prev = 2 (for operator)
  li $t0, 2
  j IsvalidLoop

end_loop:
  # if previous was '(', return false
  beq $t0, 3, unbalanced
  # if previous was an operator, return false
  beq $t0, 2, unbalanced
  # return 1 (true)
  li $v0, 1
  j end_expression

unbalanced:
  # return 0 (false)
  li $v0, 0

end_expression:
  lw $ra, 0($sp)
  lw $t0 , 4($sp)
  lw $t1 , 8($sp)
  lw $t2, 12 ($sp)
  addi $sp , $sp 16
  jr $ra
```

```cpp

bool isValidExpression(string s)
{
    if(!Balance(s))return false;
    int prev=-1;      // 1 for digits , 2 for operators , 3 for '('  , 4 for ')'
    for(char c:s){
        if(c=='(')
        {
            if(prev==4 || prev==1)return false;
            prev=3;
        }
        else if(c==')')
        {
            if(prev==-1 || prev==3)return false;
            prev=4;
        }else if(isdigit(c))
        {
            if(prev==4)return false;
            prev=1;
        }else if(c=='-'  || c=='+'  || c=='*' || c=='/')
        {
            if(prev==-1 || prev==2 ||  prev==3)return false;
            prev=2;
        }
    }
    if(prev==3 || prev==2)
        return false;
    return true;
}
```

Mips

```
isValidExpression:
    addi $sp, $sp, -16
    sw $ra, 0($sp)
    sw $t0, 4($sp)
    sw $t1, 8($sp)
    sw $t2, 12($sp)

    # &standardExp
    move $t2 , $a0
    # flag for the previous char
    # 1-->digit, 2-->operator
    # 3-->'(',   4-->')'
    li $t0, -1

    IsvalidLoop:
      # load from standardExp
      lb $t1, 0($t2)
      # if('\n') break
      beq $t1, '\n', end_loop
      # if(null-terminator) break
      beqz $t1, end_loop
      # moving to next character
      addi $t2, $t2, 1

      # Check character
      beq $t1, '(', check_open_bracket
      beq $t1, ')', check_close_bracket

      # call isDigit function    $v0 return flage
      move $a1 , $t1
      jal isDigit        # check if character is a digit
      bne $v0, $zero, checkDigit
```

```
    # call is operator function
    move $a0 , $t1
    jal isOperator
    # if not a digit, check if it's an operator
    bne $v0, $zero, check_operator

  check_open_bracket:
    # if previous was ')' or initial, return false
    beq $t0, 4, unbalanced
    # if previous was a digit, return false
    beq $t0, 1, unbalanced
    # set prev = 3 (for '(')
    li $t0, 3
    j IsvalidLoop

  check_close_bracket:
    # if previous was initial, return false
    beq $t0, -1, unbalanced
    # if previous was '(', return false
    beq $t0, 3, unbalanced
    # if previous was operator , return false
    beq $t0, 2, unbalanced
    # set prev = 4 (for ')')
    li $t0, 4
    j IsvalidLoop
checkDigit:
    beq $t0, 4, unbalanced
    li $t0, 1
    j IsvalidLoop
```

```
check_operator:
    # if previous was initial, return false
    beq $t0, -1, unbalanced
    # if previous was an operator, return false
    beq $t0, 2, unbalanced
    # if previous was '(', return false
    beq $t0, 3, unbalanced
    # set prev = 2 (for operator)
    li $t0, 2
    j IsvalidLoop

end_loop:
    # if previous was '(', return false
    beq $t0, 3, unbalanced
    # if previous was an operator, return false
    beq $t0, 2, unbalanced
    # return 1 (true)
    li $v0, 1
    j end_expression

unbalanced:
    # return 0 (false)
    li $v0, 0

end_expression:
     lw $ra, 0($sp)
     lw $t0 , 4($sp)
     lw $t1 , 8($sp)
     lw $t2, 12 ($sp)
     addi $sp , $sp 16
     jr $ra
```

Balance

c

```c
bool Balance(string s)
{
    int cnt=0;
    for(char c:s)
    {
        if(c=='(')
            cnt++;
        else if(c==')')
            cnt--;
        if(cnt<0)
            return false;
    }
    if(cnt)
        return false;
    return true;
}
```

Mips

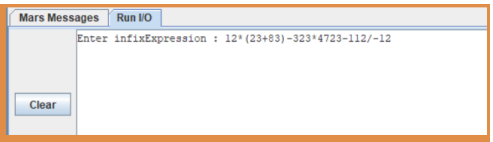
```mips
Balance:
    addi $sp, $sp, -4
    sw $ra, 0($sp)

    # counter i
    li $t0, 0
    # Loop over the string
    loop2:
    # load from standardExpression
    lb $t1, 0($a0)
    # if(null-terminator) break
    beqz $t1, end_loop2
    # moving to the next character
    addi $a0, $a0, 1
    # increment counter if '('
    beq $t1, '(', increase_cnt
    # decrement counter if ')'
    beq $t1, ')', decrease_cnt
    # loop
    j loop2

    increase_cnt:
    # increment counter
    addi $t0, $t0, 1
    j loop2
```

```mips
decrease_cnt:
# decrement counter
addi $t0, $t0, -1
# if(counter < 0) return 0
bltz $t0, unbalanced2
j loop2

unbalanced2:
# return 0
li $v0, 0
j end_balance2

end_loop2:
# if(counter != 0) return 0
bnez $t0, unbalanced2
# else return 1
li $v0, 1

end_balance2:
   lw $ra, 0($sp)
   addi $sp, $sp, 4
   jr $ra
```

5. Main Function: -

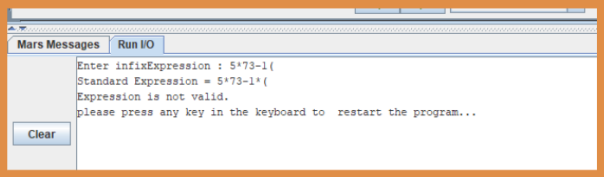
```
.data
# Global variables
    filteredExp:              .space 1200               # filtered expression
    standardExp:              .space 1200               # Assuming res array
    minusMul:                 .asciiz  "(0-1)*"
    minusDiv:                 .asciiz  "(0-1)/"
    fail_msg:                 .asciiz "Expression is not valid.\n"
    restart:                  .asciiz  "please press any key in the keyboard to  restart the program..."
    restart_done:             .asciiz "Restart Done\n"
    head:                     .word 0                 # Pointer to the head of the linked list (initialized to null)  # integer stack
    stack_size:               .word 0                   # Size of the stack (initialized to 0)   #integer stack
    array_size:               .word 0
    string_array:             .space 1000
    tempStringSize:           .word 0
    newLine:                      .asciiz "\n"
    space:                    .asciiz " "
    tempString:               .space 36
    expression:               .space 1200
    tempExpression:           .space 1200
    emptyStack:               .asciiz "Stack is empty\n"
    prompt:                   .asciiz "Enter infixExpression : "
    valuPrint:                .asciiz "Value: "
    POS_Str:                  .asciiz "postfix Expression = "
    STANDARD_STR:          .asciiz "Standard Expression = "
```

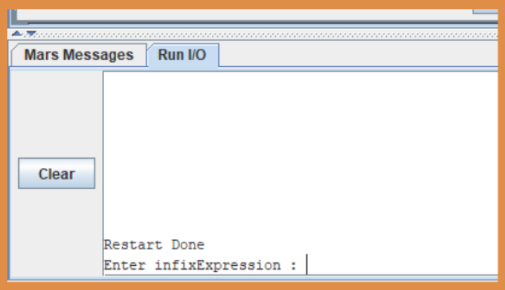
```
main:
        la $a0 , Promote
        jal printString

        la $a0 , tempexpression
        li $a1 , 100
        li $v0 , 8
        syscall

        la $t0 , tempexpression
        la $t1 , s
        li $t3 , ' '
        li $t4 , '\n'
        Filterloop:
              lb $t2 , 0($t0)
              beq $t2 , $zero , null
              beq $t2 , $t3 , incrementFlterLoop
              beq $t2 , $t4 , incrementFlterLoop
              sb $t2 , 0($t1)
              addi $t1 , $t1 , 1
              incrementFlterLoop:
                   addi $t0 , $t0 , 1
                   j Filterloop

              null:
                sb $t2 , 0($t1)
                j exiteFilterLoop

        exiteFilterLoop:
```

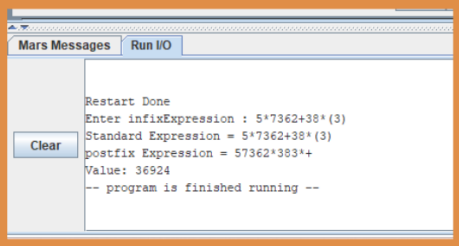



```
        exiteFilterLoop:

        jal to_standard_infinix
        la $a0, STANDARD_STR        # Load address of res
        jal printString

    la $a0, res        # Load address of res
    jal printString

        jal printNewLine

     # go to validation

      # Load address of the string into $a0
      la $a0, res

      # Call the Bal function to check parentheses balance
        jal Bal
        beq $v0,$0,not_valid
     # Call the isValidExpression function
    la $a0 , res
        jal isValidExpression

        # Check the result
       bne $v0, $zero, valid
```

```
82
83      # Check the result
84     bne $v0, $zero, valid
85  not_valid:
86      la $a0 , fail_msg
87      jal printString
88
89      la $a0 , restart
90      jal printString
91
92      li $v0 , 12   # system call for read char
93      syscall
94      li $t0 , 0
95      li $t1 ,20
96
97  newLineLOOP:
98      beq $t0 , $t1 ,  exite_newLineLOOP
99
100     jal printNewLine
101
102     addi $t0 , $t0 ,1
103     j newLineLOOP
104
105 exite_newLineLOOP:
106
107
108     la $a0 , restart_done
109     jal printString
110
111     j main
```





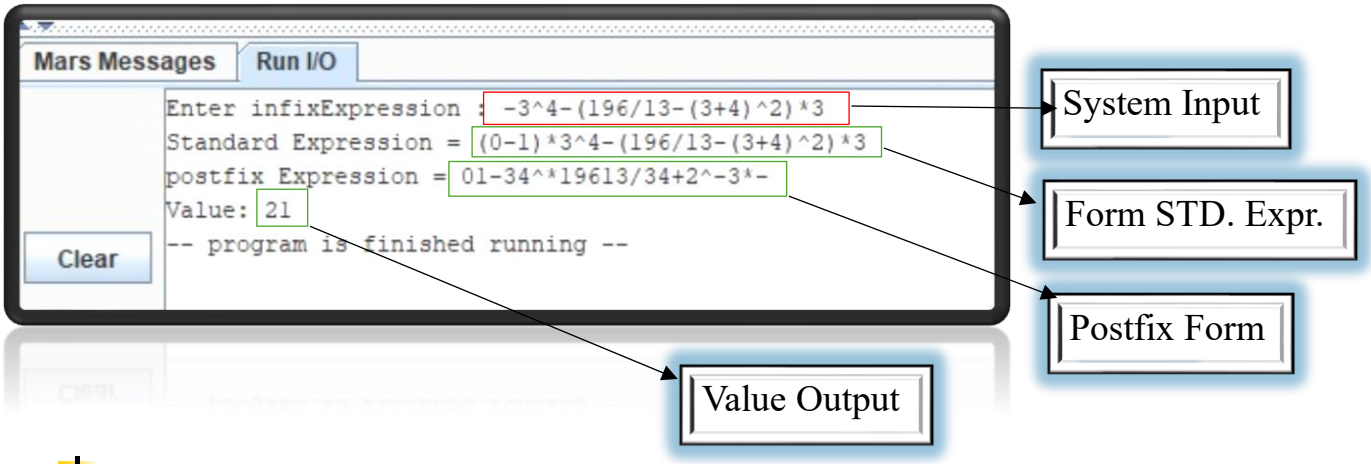



```
112
113   valid:
114
115     la $t0 , res
116     la $t1 , expression
117
118     copyloop1:
119           lb $t2 , 0($t0)
120           beq $t2 , $zero , null12
121
122           sb $t2 , 0($t1)
123           addi $t1 , $t1 , 1
124
125           addi $t0 , $t0 , 1
126           j copyloop1
127           null12:
128             sb $t2 , 0($t1)
129             j exitecopyloop1
130
131      exitecopyloop1:
132
133     jal Infix_to_postfix
134
135        la $a0 , POS_Str
136        jal printString
137
138        la $s4, string_array
139     li $s5, 0
140     lw $t6, array_size
141
142     print_postfix_loop:
143           bge $s5, $t6, end_print_postfix_loop
144        sll $t7, $s5, 2
145        add $t7, $t7, $s4
146           lw $t8, 0($t7)
147
148           move $a0, $t8
149               jal printString
150
151              addi $s5, $s5, 1
152              j print_postfix_loop
153        end_print_postfix_loop:
```

```
153                 end_print_postfix_loop:
154                     jal printNewLine
155
156                     la $a0 , string_array
157                     jal Evaluation
158
159                     # move result in  $t0
160                     move $t0 , $v0
161                         la $a0 , valuPrint
162                         jal printString
163
164                  li $v0 , 1
165                  move $a0 , $t0
166                 syscall
167
168                 li $v0, 10 # end program
169                 syscall
170   #end of main function
```

Test Cases

Sample [Input/Output]:

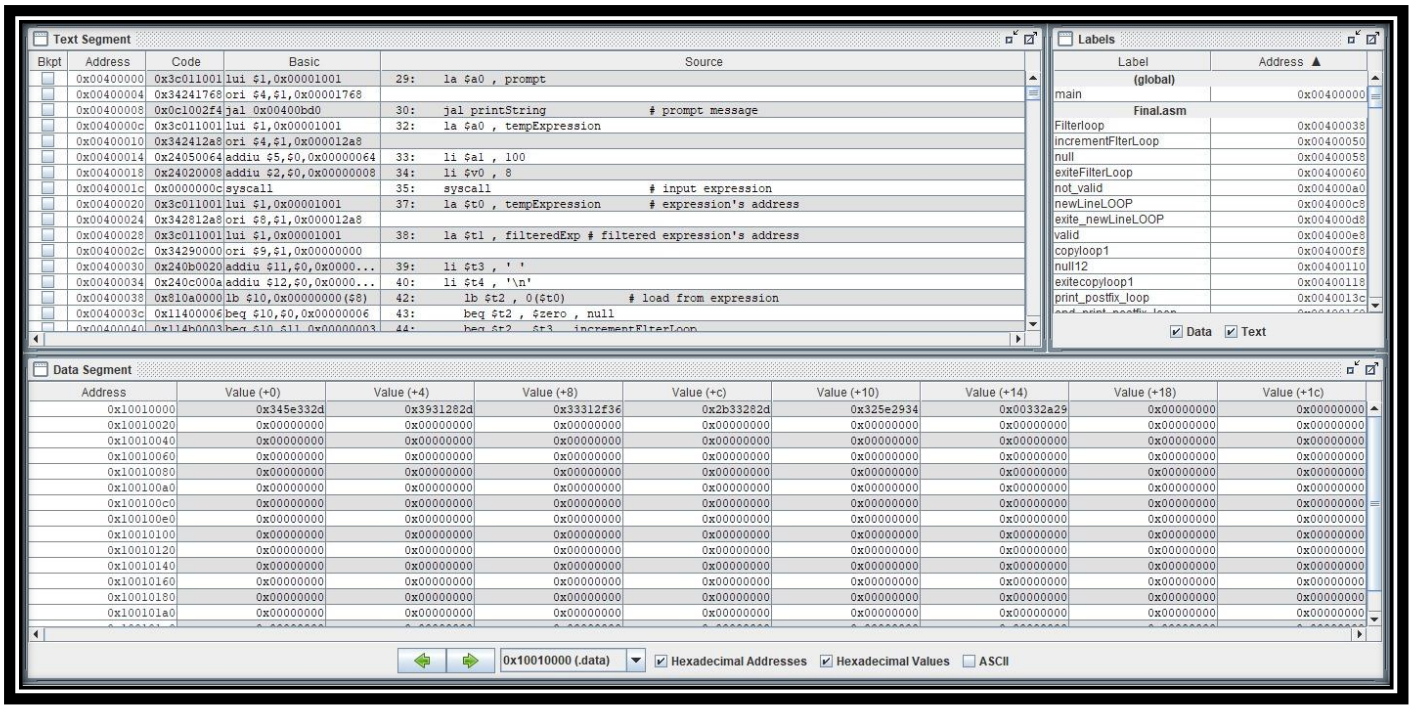


Register Values:

Registers | Coproc 1 | Coproc 0

Name	Number	Value
$zero	0	0x00000000
$at	1	0x10010000
$v0	2	0x0000000a
$v1	3	0x00000000
$a0	4	0x00000015
$a1	5	0xffffff9a
$a2	6	0x0000002d
$a3	7	0x00000000
$t0	8	0x00000015
$t1	9	0x10010dd5
$t2	10	0x00000000
$t3	11	0x00000013
$t4	12	0x0000001b
$t5	13	0x00000000
$t6	14	0x00000013
$t7	15	0x00000013
$s0	16	0x00000001
$s1	17	0x0000001c
$s2	18	0x10010a30
$s3	19	0xffffff9a
$s4	20	0xffffffaf
$s5	21	0x00000013
$s6	22	0x00000000
$s7	23	0x00000000
$t8	24	0x10040138
$t9	25	0x10040138
$k0	26	0x00000000
$k1	27	0x00000000
$gp	28	0x10008000
$sp	29	0x7fffeffc
$fp	30	0x00000000
$ra	31	0x00400180
pc		0x00400194
hi		0xffffffff
lo		0xffffff9a

Memory Segment Values:



Extra Examples:

```
Enter InfixExpression : 2^3-4+4^2+4*-3
Standard Expression   : 2^3-4+4^2+4*(0-1)*3
postfix Expression    : 23^4-42^+401-*3*+
Value                 : 8
-----------------------------------------------------------------------------------------------------------------------
Enter infixExpression : 5*(12+8)-3*(4-2)
Standard Expression   : 5*(12+8)-3*(4-2)
postfix Expression    : 5128+*342-*-
Value                 : 94
-----------------------------------------------------------------------------------------------------------------------
Enter InfixExpression : -23*837*(-12/-(2*-5)++9-89)*3/12*72+(-1*23-+2)*4(4--2)(4++2)
Standard Expression   : (0-1)*23*837*((0-1)*12/(0-1)/(2*(0-1)*5)+9-89)*3/12*72+((0-1)*1*23-2)*4*(4-(0-1)*2)*(4+2
postfix Expression    : 01-23*837*01-12*01-/201-*5*/9+89-*3*12/72*01-1*23*2-4*401-2*-*42+*+
Value                 : 28064304
 -----------------------------------------------------------------------------------------------------------------------
Enter InfixExpression : -5*1234*(-8/-(3*-6)+7-56)*4/7*89+(-2*45-+8)*3*(6--3)*(6++3)
Standard Expression   : (0-1)*5*1234*((0-1)*8/(0-1)/(3*(0-1)*6)+7-56)*4/7*89+((0-1)*2*45-8)*3*(6-(0-1)*3)*(6+3)
postfix Expression    : 01-5*1234*01-8*01-/301-*6*/7+56-*4*7/89*01-2*45*8-3*601-3*-*63+*+
Value                 : 15351826
-----------------------------------------------------------------------------------------------------------------------
Enter InfixExpression : 2^4-3*2*(3+3)/-4(4+5)(5--4)+(-4)^2
Standard Expression   : 2^4-3*2*(3+3)/(0-1)/4*(4+5)*(5-(0-1)*4)+((0-1)*4)^2
postfix Expression    : 24^32*33+*01-/4/45+*501-4*-*-01-4*2^+
Value                 : 761
-----------------------------------------------------------------------------------------------------------------------
```

Task Assignments

Each member participated in the project with: -

	Searching and writing code	Reviewing code	Code compilation	Report	presentation
Abdelrahman Mahmoud Mohamed	**16.67%**	**16.67%**	**16.67%**	**16.67%**	**16.67%**
Ahmed Bahy Youssef	**16.67%**	**16.67%**	**16.67%**	**16.67%**	**16.67%**
Ahmed Hossam Mohamed Fekry	**16.67%**	**16.67%**	**16.67%**	**16.67%**	**16.67%**
Ahmed Ragab Mahmoud	**16.67%**	**16.67%**	**16.67%**	**16.67%**	**16.67%**
Ahmed Sobhy Refaay	**16.67%**	**16.67%**	**16.67%**	**16.67%**	**16.67%**
Hassan Hussien Azmy	**16.67%**	**16.67%**	**16.67%**	**16.67%**	**16.67%**